

DAY 2

Toolchain: Python, VS Code, Jupyter, OpenCV, NumPy, Matplotlib & Git

Topics covered: *Free toolchain setup · Python & pip · VS Code · Virtual environments · Jupyter/Colab · OpenCV · NumPy · Matplotlib · Git*

SECTION 1 — THEORY

1.1 Why a Toolchain Matters

Before writing a single line of computer vision or GenAI code, you need a working environment: a Python interpreter, an editor, a way to run and iterate on code interactively, a set of libraries for handling images and data, and a way to track your work over time. Today's focus is entirely free and open-source — every tool below has a no-cost tier that is more than enough to build real projects.

1.2 Python & Package Management

Python is the primary language for this course. To get started, install a specific or the newest stable version of Python for your operating system.

- A Python project is built from modules and packages. A module is a single .py file of reusable code; a package is a folder of related modules. Both can be user-defined (written by you) or predefined (published by others).
- The import statement is how you pull a module or package into your current script so you can use its functions and classes.
- Predefined (third-party) packages must be installed before you can import them. PyPI (the Python Package Index) is the central repository where virtually every publicly available Python package is hosted — you can search it for any package you need.

The standard tool for installing packages from PyPI is pip. The recommended way to invoke it is as a Python module, using the -m flag:

```
(venv) :~$ python -m pip install arrow
```

- This command runs pip as a module and downloads/installs the arrow package.
- Using python -m ensures the pip version that matches your active Python interpreter is used — calling pip directly can sometimes invoke a pip binary tied to a different, incompatible Python installation.

The same pattern is used to remove a package:

```
(venv) :~$ python -m pip uninstall arrow
```

- Python's official documentation (python.org) is the authoritative reference for every valid form of the python and pip commands.

1.3 VS Code as a Development Environment

Visual Studio Code (VS Code) is a general-purpose code editor (IDE) — it is not tied to Python and supports many languages, including JavaScript. To write and run Python inside VS Code, install Microsoft's official Python extension first; this adds Python-aware features like linting, debugging, and interpreter selection.

By default, VS Code points to whatever Python installation is set as the system's global interpreter, shown at the bottom of the editor. For real projects, you should instead create and use a dedicated virtual environment.

1.4 Virtual Environments — Why Isolate Dependencies

A virtual environment is an isolated, self-contained copy of Python and its installed packages, kept separate from your system-wide Python installation and from other projects. This matters because different projects often need different, sometimes conflicting, versions of the same package.

- `venv` — Python's built-in tool; creates a lightweight `.venv` folder scoped to the current project/workspace.
- `conda` — an alternative environment and package manager (via Anaconda/Miniconda) that also manages non-Python dependencies; creates a `.conda` environment.

Inside VS Code, a virtual environment is created through the Command Palette:

1. Open the Command Palette (top of the editor, or Show and Run Commands).
2. Run “Python: Create Environment...”
3. Choose the environment type — Venv (creates a `.venv` in the current workspace) or Conda (creates a `.conda` environment).
4. Select which installed Python version/interpreter the new environment should be built from.

1.5 Notebooks: Jupyter vs Google Colab

Notebooks let you run code in small, independent, re-runnable cells and see output — including images and plots — inline, which makes them ideal for exploratory CV and data work.

- Jupyter (local, inside VS Code): install the Jupyter extension from the Extensions marketplace; once installed, you can create a native notebook file with the `.ipynb` extension and run cells directly in the editor.
- Google Colab (cloud, free tier): a hosted notebook environment that runs in the browser with no local setup, and includes free access to shared compute — a convenient alternative when you don't want to configure a local environment.

1.6 Core CV & Data Libraries

1.6.1 OpenCV

- OpenCV (Open Source Computer Vision Library) is an open-source library for computer vision, image processing, and machine learning.
- It offers a comprehensive toolkit for analyzing, modifying, and understanding images and video, widely used in both research and industry.
- It supports a broad range of CV tasks — image processing, object detection, feature extraction — and provides interfaces in multiple languages, including Python, C++, and Java.

1.6.2 NumPy

- NumPy is a core Python package that behaves like a module once imported into your code.
- It is a foundational tool for scientific computing, data analysis, and machine learning, and is central to how images are represented in computer vision.
- Its key feature is the ndarray — a highly optimized N-dimensional array object that is far more efficient than a standard Python list for numerical work.

To use it, import it at the top of your code (by convention, aliased as np):

```
import numpy as np
```

- NumPy ships as part of a standard Python environment, so no separate installation is usually required.

Recall from Day 1 that an image is just a digital grid of numbers (pixels). NumPy is what lets you hold and manipulate that grid programmatically — but NumPy itself cannot open a .jpg or .png file directly. It needs to be paired with an image-loading library such as Pillow (PIL) to read the file into an array in the first place.

Reference: the full NumPy source and API live in its official GitHub repository ([numpy/numpy](https://numpy.org)).

1.6.3 Matplotlib

- Matplotlib is a Python library for building static, interactive, and animated visualizations from data.
- Its flexible, customizable plotting functions make it easy to understand patterns, trends, and relationships in data.
- In computer vision specifically, Matplotlib is the library most commonly used to plot and display images once they're loaded as arrays.

```
import matplotlib.pyplot as plt
```

Older notebook setups required the magic command `%matplotlib inline` to render plots directly inside the notebook. In current versions of Jupyter, inline plotting works by default and this line is no longer necessary.

1.7 Git & Version Control

- Git is a free, open-source, distributed version control system for tracking changes to files and coordinating work across multiple people.
- Conceptually, it behaves like a checkpoint-save system for a project: you can record history, revert to any previous version, and collaborate without losing data.
- A useful mental model: Git is like an onion — the commands you use daily are just the outer layers; peeling them back reveals the simpler, lower-level mechanics underneath.

Git's commands split into two tiers:

Porcelain commands — everyday, user-facing commands

Command	What it does
git add	Stages changes (moves them from working directory into the staging area) so they are ready to be committed.
git commit	Saves a permanent checkpoint of the staged changes to the project's local history.
git push	Uploads local commits to a remote repository (e.g., GitHub) so others can see them.

Command	What it does
git pull	Downloads and merges changes from a remote repository into the local branch.
git branch	Lists, creates, or deletes branches — independent lines of development.
git switch	Moves you to a different existing branch.
git merge	Combines the changes from one branch into another.
git rebase	Re-applies commits from one branch on top of another, producing a cleaner, linear history.

Plumbing commands — low-level commands porcelain is built on

Command	What it does
git cat-file	Inspects the raw content and type of any object stored in Git's internal database.
git hash-object	Computes and optionally stores the unique hash (ID) Git assigns to a piece of content.
git count-objects	Reports how many objects exist in the repository and how much space they consume.

A concise, formal definition worth remembering: Git is a distributed revision control system.

Key Takeaways — Section 1

- Always install packages with “python -m pip install/uninstall <package>” to guarantee pip matches your active interpreter.
- Isolate every project in its own virtual environment (venv or conda) to avoid dependency conflicts.
- Jupyter (local) and Colab (cloud) both give you cell-by-cell, inline-output notebooks — pick based on whether you want local control or zero setup.
- OpenCV handles CV algorithms, NumPy represents images as arrays, and Matplotlib visualizes them — the three form a core trio for CV work.
- Git tracks project history through everyday “porcelain” commands (add, commit, push, branch...), which are themselves built on lower-level “plumbing” commands.

SECTION 2 — ARCHITECTURE & REAL-WORLD USE CASES

2.1 The End-to-End Toolchain Architecture

Individually, each tool from Section 1 solves one piece of the puzzle. Wired together, they form the standard local/cloud toolchain you'll use for every CV and GenAI project in this course:

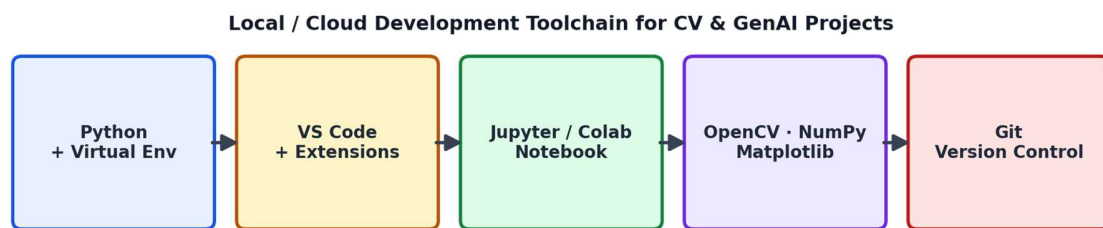


Fig 2.1 — Python + a virtual environment is edited in VS Code, executed interactively in a Jupyter/Colab notebook, powered by OpenCV/NumPy/Matplotlib for CV and data work, with Git tracking every change.

2.2 Setting Up a Python Virtual Environment in VS Code

This is the practical, repeatable workflow you'll use at the start of every new project:

- 1. Open the Command Palette** — Click at the very top of VS Code (or use the keyboard shortcut) to open the searchable command bar.
- 2. Run “Show and Run Commands”** — This opens the full list of available editor and extension commands.
- 3. Select “Python: Create Environment...”** — This starts VS Code's guided environment-creation flow.
- 4. Choose “Venv”** — This tells VS Code to create a lightweight .venv virtual environment scoped to the current workspace (Conda is the alternative if you need non-Python dependencies too).
- 5. Pick the base Python interpreter** — Select which installed Python version (e.g., 3.13, 3.12, 3.10...) the new environment should be created from.

Once created, VS Code automatically switches the active interpreter shown at the bottom of the editor to your new project-local environment.

2.3 Installing & Using Jupyter Notebooks in VS Code

- 1. Open the Extensions panel** — Search for “Jupyter” and install Microsoft's official Jupyter extension — it adds notebook support and interactive, cell-based execution directly inside VS Code.
- 2. Create a notebook file** — Once the extension finishes installing, create a new file with the .ipynb extension — VS Code will open it in the native notebook interface, ready to run Python cell by cell.

2.4 Real-World Mini Pipeline: Loading, Representing & Displaying an Image

This ties Day 1's theory (an image is a numeric pixel grid) directly to the tools from today. A minimal but complete CV “hello world” combines Pillow (to read the file), NumPy (to hold it as an array), and Matplotlib (to display it):

```
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt

img = Image.open("sample.jpg")      # load the image file
pixel_grid = np.array(img)          # convert to a NumPy H x W x C array
print(pixel_grid.shape)              # e.g. (height, width, 3) for RGB

plt.imshow(pixel_grid)               # render the array as an image
plt.axis("off")
plt.show()
```

This four-line pattern — load, array, inspect shape, display — is the backbone of nearly every CV notebook you will write in this course.

2.5 A Real-World Git Workflow for a CV/GenAI Project

Applying the porcelain commands from Section 1 to a typical feature-branch workflow:

1. `git branch feature/image-loader` — create a new branch for the feature you're building.
2. `git switch feature/image-loader` — move into that branch to work in isolation.
3. `git add .` — stage the files you've changed (e.g., your new notebook or script).
4. `git commit -m "Add image loading pipeline"` — save a checkpoint with a descriptive message.
5. `git push` — upload the branch and its commits to the remote repository (e.g., GitHub).
6. `git merge` or `git rebase` — once reviewed, integrate the feature branch back into main, choosing merge to preserve full branch history or rebase for a clean, linear log.

This same five/six-step rhythm — branch, switch, add, commit, push, merge — scales from a solo learning project to a full production ML/GenAI codebase.

Key Takeaways — Section 2

- The toolchain has a fixed shape: Python+venv → VS Code → Notebook → CV/data libraries → Git, regardless of the project.
- Creating a virtual environment and installing Jupyter in VS Code are both driven through the Command Palette — a pattern worth memorizing.
- Image + NumPy + Matplotlib (“load → array → display”) is the minimal working pipeline behind almost every CV notebook.
- A branch → switch → add → commit → push → merge/rebase cycle is the real-world Git workflow you'll repeat for every feature, in every project, going forward.

Sources & Further Reading

- Python — <https://app.pluralsight.com/ilx/video-courses/python-development-environments/course-overview>
- Vs code — <https://app.pluralsight.com/library/courses/amazon-bedrock-using-python>
- Jupyter — <https://app.pluralsight.com/library/courses/amazon-bedrock-using-python>
- OpenCV — <https://www.udemy.com/course/python-for-computer-vision-with-opencv-and-deep-learning/learn/lecture/12291386#overview>
- NumPy — <https://www.udemy.com/course/python-for-computer-vision-with-opencv-and-deep-learning/learn/lecture/12278158#overview>
- Matplotlib — <https://www.udemy.com/course/python-for-computer-vision-with-opencv-and-deep-learning/learn/lecture/12257486#overview>
- Git — <https://app.pluralsight.com/ilx/video-courses/how-git-works/course-overview>